

RAPORT:

Proiect - Clasificarea boabelor uscate de fasole folosind Spark ML

Cuprins

1. Problemă și obiectiv
2. Date: sursă, dimensiune, schemă, calitate, limitări
3. Arhitectură: fluxul de date și tehnologiile folosite
4. Implementare: transformări principale și decizii importante
5. Rezultate: valori, grafice, tabele sau modele obținute
6. Benchmark: scenarii, configurații, metrici și interpretare
7. Reproductibilitate: pași de rulare și variabile de mediu
8. Limitări și următorul pas
9. Concluzii
10. Declarație AI
11. Bibliografie

1. Problemă și obiectiv

Prelucrarea volumelor mari de date presupune nu doar citirea și transformarea datelor, ci și construirea unor fluxuri de lucru reproductibile, capabile să proceseze datele și să genereze rezultate măsurabile.

În acest proiect a fost realizat un experiment de clasificare, rulat local prin intermediul Visual Studio Code, cât și în cloud prin intermediul mediului cloud gestionat pentru Spark Databricks, folosind Apache Spark și biblioteca Spark ML. Problema abordată este clasificarea tipurilor de boabe uscate de fasole pe baza unor caracteristici numerice extrase de specialiști din imagini.

Obiectivul proiectului este construirea unui pipeline de machine learning în Spark ML, care să includă încărcarea datelor, verificarea structurii datasetului, pregătirea caracteristicilor, împărțirea datelor în set de antrenare și set de test, antrenarea mai multor modele de clasificare și evaluarea performanței acestora.

În proiect au fost utilizați trei algoritmi din Spark ML: Decision Tree, Random Forest și Logistic Regression. Alegerea acestor algoritmi permite compararea unui model bazat pe un singur arbore de decizie, a unui model ensemble bazat pe mai mulți arbori și a unui model liniar de clasificare.

Deoarece notebook-urile din workspace-ul Databricks pot fi accesate doar de utilizatori înregistrați în același workspace, varianta rulată în cloud este inclusă în arhiva proiectului ca export .html de unde se poate descărca. Link pentru notebook Databricks: [notebook Databricks](#).

Link github: [Proiect_PVMD](#)

2. Date: sursă, dimensiune, schemă, calitate, limitări

Datasetul utilizat este Dry Bean Dataset[1]. Acesta conține informații despre boabe uscate de fasole, obținute printr-un sistem de computer vision. Pentru fiecare boabă au fost extrase caracteristici numerice legate de formă și dimensiune.

Datasetul are 13.611 instanțe, 16 caracteristici numerice și o coloană țintă numită Class. Coloana Class reprezintă tipul de fasole care trebuie prezis de model.

Clasele existente în dataset sunt:

BARBUNYA, BOMBAY, CALI, DERMAISON, HOROZ, SEKER și SIRA.

Caracteristicile numerice folosite în proiect sunt:

Area, Perimeter, MajorAxisLength, MinorAxisLength, AspectRatio, Eccentricity, ConvexArea, EquivDiameter, Extent, Solidity, roundness, Compactness, ShapeFactor1, ShapeFactor2, ShapeFactor3 și ShapeFactor4.

Din punct de vedere al calității datelor, datasetul este potrivit pentru o problemă de clasificare supravegheată, deoarece conține valori numerice clare și o etichetă de clasă pentru fiecare instanță. Totuși, datasetul nu include probleme complexe precum text nestructurat, valori lipsă masive sau date distribuite pe mai multe surse. Din acest motiv, proiectul este mai ușor de urmărit și de înțeles.

```
... Distribuția claselor:
+-----+-----+
| Class|count|
+-----+-----+
| CALI| 1630|
| SEKER| 2027|
| SIRA| 2636|
| HOROZ| 1928|
| BOMBAY| 522|
| BARBUNYA| 1322|
| DERMAISON| 3546|
+-----+-----+

Total rânduri: 13611
Numar total de coloane: 17
```

```
feature_cols = [c for c in df.columns if c != "Class"]

print("coloane folosite ca features:")
print(feature_cols)

[29] ✓ 0.0s

... Coloane folosite ca features:
['Area', 'Perimeter', 'MajorAxisLength', 'MinorAxisLength', 'AspectRatio', 'Eccentricity', 'ConvexArea', 'EquivDiameter', 'Extent', 'Solidity', 'roundness']
```

3. Arhitectură: fluxul de date și tehnologiile folosite

Implementarea proiectului a fost realizată într-un Jupyter Notebook rulat local în Visual Studio Code. A fost creat și un notebook python în databricks pentru rularea în cloud a celor 3 algoritmi. Pentru procesarea datelor și antrenarea modelului a fost utilizat PySpark, împreună cu biblioteca Spark ML.

Fluxul de date al proiectului este următorul:

Datele sunt încărcate din fișierul Dry_Bean_Dataset.csv. După încărcare, este verificată structura datasetului, inclusiv coloanele existente, numărul de rânduri și distribuția claselor. Apoi sunt selectate coloanele numerice care vor fi folosite ca variabile de intrare pentru model.

Coloana țintă Class, care este de tip text, este transformată într-o etichetă numerică folosind StringIndexer. Caracteristicile numerice sunt combinate într-un singur vector de features folosind VectorAssembler. După aceea, datele sunt împărțite în set de antrenare și set de test, în proporție de 80% pentru antrenare și 20% pentru testare.

Modelele de clasificare sunt integrate în pipeline-uri Spark ML, împreună cu pașii de preprocesare. Pentru fiecare algoritm, pipeline-ul este format din trei etape: transformarea etichetei text în label numeric, construirea vectorului de caracteristici și antrenarea modelului. Algoritmii evaluați sunt Decision Tree, Random Forest și Logistic Regression.

Fiecare model este antrenat pe setul de train, iar predicțiile sunt generate pe setul de test. La final, modelele sunt evaluate folosind metricile Accuracy și F1-score, iar timpul de antrenare este măsurat pentru fiecare algoritm.

Tehnologiile folosite în proiect sunt Apache Spark, PySpark, Spark ML, Jupyter Notebook și Visual Studio Code pentru rulare locală și Databricks pentru rulare în cloud.

4. Implementare: transformări principale și decizii importante

Implementarea proiectului a urmat un flux clar de lucru. Primul pas a fost importarea librăriilor necesare și pornirea unei sesiuni Spark. Apoi a fost încărcat fișierul CSV care conține datasetul Dry Bean.

După încărcarea datelor, au fost verificate structura datasetului, distribuția claselor și numărul total de rânduri și coloane. Au fost alese toate coloanele numerice drept caracteristici de intrare, iar coloana Class a fost păstrată ca variabilă țintă.

O decizie importantă a fost folosirea unui pipeline Spark ML, deoarece acesta permite legarea tuturor pașilor într-o singură structură reproductibilă. Pentru fiecare algoritm, pipeline-ul conține trei componente principale: transformarea etichetei text în etichetă numerică, combinarea caracteristicilor într-un vector și antrenarea modelului de clasificare.

Pentru transformarea coloanei Class a fost folosit StringIndexer, care convertește clasele textuale în valori numerice. Pentru combinarea caracteristicilor numerice a fost folosit VectorAssembler, care creează coloana features, necesară pentru algoritmi din Spark ML.

Modelele folosite au fost:

- DecisionTreeClassifier
- RandomForestClassifier
- LogisticRegression

Decision Tree este un model bazat pe un singur arbore de decizie. Random Forest extinde această idee prin construirea mai multor arbori și combinarea predicțiilor acestora. Logistic Regression este un model liniar folosit pentru clasificare și a fost inclus ca metodă de comparație.

Parametrii principali folosiți au fost:

- Decision Tree: maxDepth = 6, seed = 42
- Random Forest: numTrees = 30, maxDepth = 6, seed = 42
- Logistic Regression: maxIter = 50

```
label_indexer = StringIndexer(  
    ... inputCol="class",  
    ... outputCol="label"  
)  
  
assembler = VectorAssembler(  
    ... inputCols=feature_cols,  
    ... outputCol="features"  
)  
  
dt = DecisionTreeClassifier(  
    ... featuresCol="features",  
    ... labelCol="label",  
    ... maxDepth=6,  
    ... seed=42  
)  
  
rf = RandomForestClassifier(  
    ... featuresCol="features",  
    ... labelCol="label",  
    ... numTrees=30,  
    ... maxDepth=6,  
    ... seed=42  
)  
  
lr = LogisticRegression(  
    ... featuresCol="features",  
    ... labelCol="label",  
    ... maxIter=50  
)
```

5. Rezultate: valori, grafice, tabele sau modele obținute

În urma rulării proiectului, cele trei modele Spark ML au fost antrenate pe 10.953 de rânduri și testate pe 2.658 de rânduri. Modelele evaluate au fost Decision Tree, Random Forest și Logistic Regression.

Rezultatele obținute sunt prezentate în tabelul de benchmark generat în notebook. Pentru fiecare algoritm au fost calculate Accuracy, F1-score și timpul de antrenare.

```

=====
Model: Decision Tree
Accuracy: 0.9011
F1-score: 0.9008
Timp antrenare: 1.7890 secunde

+-----+-----+-----+-----+
|class  |label|prediction|probability|
+-----+-----+-----+-----+
|HOROZ  |3.0  |3.0       |[0.0,7.722007722007722E-4,0.0,0.9915057015057915,0.0060498069498069494,7.722007722007722E-4,0.0]|
|SIRA   |1.0  |1.0       |[0.034837235865210876,0.9194745859508852,0.013706453455168474,0.013706453455168474,0.009708737864077669,0.008566533409480296,0.0]|
|BARBUNYA|5.0  |4.0       |[0.0,0.116883116883117E-4,0.0,0.00974025974025974,0.899356483506493,0.08047402597402597,0.0016233766233766235]|
|DERMASON|0.0  |0.0       |[0.9838187702265372,0.01155802126675913,0.003236245954692557,0.0013869625520110957,0.0,0.0,0.0]|
|BARBUNYA|5.0  |5.0       |[0.0,0.0014265335235378032,0.007132667617689016,0.0014265335235378032,0.04850213980028531,0.9415121255349501,0.0]|
+-----+-----+-----+-----+

only showing top 5 rows
=====
Model: Random Forest
Accuracy: 0.8977
F1-score: 0.8975
Timp antrenare: 2.4322 secunde

+-----+-----+-----+-----+
|class  |label|prediction|probability|
+-----+-----+-----+-----+
|HOROZ  |3.0  |3.0       |[4.535147392290249E-5,0.003062803965971222,0.0,0.9861766706194673,0.009458242652570052,0.0012569312880684926,0.0]|
|SIRA   |1.0  |1.0       |[0.041226411167183535,0.881287152242612,0.028228350233901075,0.023977565562965025,0.005337156449666844,0.01994336434367167,0.0]|
+-----+-----+-----+-----+
...
|DERMASON|0.0  |0.0       |[0.9966998395351571,0.0013829006307751119,0.001914691512138948,2.5671937124854407E-6,1.624524279285027E-11,1.1082006535030505E-9,3.770581082692912E-12]|
|BARBUNYA|5.0  |5.0       |[3.9669114821969865E-23,1.5952460628484726E-14,8.79095102854565E-7,3.72192908410779E-13,9.929112001102507E-7,0.9999959736803193,2.1543129895353E-6]|
+-----+-----+-----+-----+

only showing top 5 rows
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

```

=====
Model: Logistic Regression
Accuracy: 0.9217
F1-score: 0.9218
Timp antrenare: 2.2249 secunde

+-----+-----+-----+-----+
|Class  |label|prediction|probability|
+-----+-----+-----+-----+
|HOROZ  |3.0  |3.0       |[1.447196387413293E-8,0.001993503549136645,4.976632836736957E-9,0.9977427986126575,2.547646848130161E-4,8.91369780296766E-6,6.993063596550066E-12]|
|SIRA   |1.0  |1.0       |[0.0017642788373981623,0.8788922834013068,0.0489006621120499,0.001247439512971397,0.030946320821149065,0.03824865472300173,3.605921230081035E-7]|
|BARBUNYA|5.0  |5.0       |[1.1150258547323002E-9,0.0012779703551988523,2.0546541701446097E-4,0.0022379963049181696,0.44465134765644015,0.5516267320941797,4.870572228465243E-7]|
|DERMASON|0.0  |0.0       |[0.9966998395351571,0.0013829006307751119,0.001914691512138948,2.5671937124854407E-6,1.624524279285027E-11,1.1082006535030505E-9,3.770581082692912E-12]|
|BARBUNYA|5.0  |5.0       |[3.9669114821969865E-23,1.5952460628484726E-14,8.79095102854565E-7,3.72192908410779E-13,9.929112001102507E-7,0.9999959736803193,2.1543129895353E-6]|
+-----+-----+-----+-----+

```

Aceste rezultate arată că modelul clasifică corect majoritatea exemplurilor din setul de test. Valorile apropiate dintre Accuracy și F1-score sugerează că performanța modelului este relativ echilibrată între clase și nu este favorabilă doar unei singure clase dominante.

Rezultatele permit compararea celor trei modele din punct de vedere al performanței și al timpului de antrenare. Decision Tree este cel mai simplu model, Random Forest este mai robust deoarece combină mai mulți arbori, iar Logistic Regression oferă o metodă liniară de comparație.

Aceeași procedură de evaluare a fost reluată și în Databricks, iar rezultatele obținute în cloud sunt prezentate mai jos:

```
=====
Model: Decision Tree
Accuracy: 0.8982
F1-score: 0.8977
Timp antrenare: 8.9781 secunde
+-----+-----+-----+-----+
|Class|label|prediction|probability|
+-----+-----+-----+-----+
|HOROZ|3.0|3.0|[0.0,0.3409090909090909,0.0,0.4886363636363636,0.10227272727272728,0.06818181818181818,0.0]|
|HOROZ|3.0|3.0|[0.0,0.06060606060606061,0.0,0.8888888888888888,0.040404040404041,0.0101010101010102,0.0]|
|BARBUNYA|5.0|5.0|[0.0,0.0014265335235378032,0.005706134094151213,0.0028530670470756064,0.0442225392296719,0.9329529243937232,0.012838801711840228]|
|DERMASON|0.0|0.0|[0.9839167455061495,0.009933774834437087,0.004730368968779565,0.0014191106906338694,0.0,0.0,0.0]|
|CALI|4.0|4.0|[0.0,7.9888450606772786F-4,0.0,0.0111731843575419,0.902633679169992,0.0,0.0,0.0,0.0]|
Model: Random Forest
Accuracy: 0.8936
F1-score: 0.8938
Timp antrenare: 7.2118 secunde
+-----+-----+-----+-----+
|Class|label|prediction|probability|
+-----+-----+-----+-----+
|HOROZ|3.0|1.0|[0.016450455194203465,0.4612454273221948,0.014345969732847826,0.361028177548585,0.09343098202428278,0.052816087192534715,6.82900985351215E-4]|
|HOROZ|3.0|3.0|[0.0038795426243566308,0.1969018817542744,0.0050513655480416515,0.7093782582445622,0.05273301807746397,0.03195384084675799,1.0209290454313427E-4]|
|BARBUNYA|5.0|5.0|[0.0,0.010200161397960683,0.009825272416728823,0.010935062812073704,0.122765911508792,0.830113225896603,0.016160365967841813]|
|DERMASON|0.0|0.0|[0.9817729299479215,0.012229364903358884,0.004499615684851033,0.0014980894638685518,0.0,0.0,0.0]|
|CALI|4.0|4.0|[0.0,0.005588979582944327,2.9532937792634977E-4,0.025383256627738566,0.8347859149488925,0.12453174463699153,0.009414774825506638]|
+-----+-----+-----+-----+

Model: Logistic Regression
Accuracy: 0.9244
F1-score: 0.9247
Timp antrenare: 11.9475 secunde
+-----+-----+-----+-----+
|Class|label|prediction|probability|
+-----+-----+-----+-----+
|HOROZ|3.0|3.0|[4.5865294103304755E-5,0.2978075079081077,5.477464860724818E-6,0.6970203075466829,0.004341939815780199,7.788987019900642E-4,3.2684749340240213E-9]|
|HOROZ|3.0|3.0|[2.405939041331441E-6,0.03319528217583258,1.4279834261657328E-11,0.9666216017182367,1.8040893773495448E-4,3.0105035159522923E-7,1.6452299430709463E-10]|
|BARBUNYA|5.0|5.0|[7.417974026275464E-9,0.003898963343567487,5.323723446668822E-6,2.080678549810924E-4,0.012372549158516134,0.9835138752083887,1.2132931259357773E-6]|
|DERMASON|0.0|0.0|[0.9992919383184358,6.332520089028653E-4,7.339547536950294E-5,1.4137540561972404E-6,4.073681355800829E-12,4.3894814932741024E-10,2.1393301513153307E-13]|
|CALI|4.0|4.0|[1.8800102372955518E-13,4.069704780172918E-6,1.1221984329351023E-7,0.001783124400990503,0.9904233300824682,0.007779865029657989,9.498562071811143E-6]|
+-----+-----+-----+-----+
```

6. Benchmark: scenarii, configurații, metrice și interpretare

Benchmark-ul a fost realizat local, în Apache Spark în modul local[*], dar și în cloud cu ajutorul Databricks. Datasetul utilizat a fost Dry_Bean_Dataset.csv, cu 13.611 instanțe, 16 caracteristici numerice și o coloană țintă.

Datele au fost împărțite în set de antrenare și set de test folosind raportul 80/20 și seed = 42. Folosirea unui seed fix permite reproducerea aceleiași împărțiri a datelor la rulări diferite.

Timpul măsurat reprezintă durata etapei de antrenare a pipeline-ului Spark ML pentru fiecare algoritm. Măsurarea include transformarea etichetei, asamblarea caracteristicilor și antrenarea modelului.

Metricile folosite pentru evaluare au fost Accuracy și F1-score. Accuracy arată proporția predicțiilor corecte din totalul predicțiilor. F1-score combină precizia și recall-ul și este util mai ales atunci când distribuția claselor nu este perfect echilibrată.

Deoarece datasetul are șapte clase, evaluarea a fost realizată cu MulticlassClassificationEvaluator.

Configurația benchmark-ului a fost:

- Algoritm: Decision Tree, Random Forest și Logistic Regression
- Mod de rulare locală: local[*]
- Mod de rulare cloud: DatabricksDataset: Dry Bean Dataset
- Rânduri totale: 13.611
- Rânduri train: 10.953
- Rânduri test: 2.658
- Accuracy: aproximativ 0.8977
- F1-score: aproximativ 0.8975
- Metrici: Accuracy, F1-score și timp de antrenare

Interpretarea benchmark-ului se face prin compararea valorilor obținute pentru cei trei algoritmi. Accuracy arată proporția predicțiilor corecte, iar F1-score oferă o imagine mai echilibrată asupra performanței modelului. Timpul de antrenare permite observarea costului de execuție pentru fiecare algoritm.

Dacă modelul Random Forest obține performanțe mai bune decât Decision Tree, acest lucru poate fi explicat prin faptul că Random Forest combină mai mulți arbori de decizie și reduce instabilitatea unui singur arbore. Logistic Regression poate avea un timp de antrenare diferit și oferă o comparație cu un model liniar.

7. Reproducibilitate: pași de rulare și variabile de mediu

Pentru rularea proiectului, fișierele `project.ipynb` și `Dry_Bean_Dataset.csv` (descărcat de la: vezi [1]) trebuie să se afle în același director. Notebook-ul poate fi rulat local în Visual Studio Code, având instalate Python și PySpark.

Pentru reproducerea rezultatelor, a fost notată și versiunea mediului de execuție. Rularea locală a fost realizată cu Apache Spark versiunea 4.2.1, iar rularea în cloud a fost realizată în Databricks, folosind runtime-ul/Spark versiunea 4.1.0. Menționarea versiunii este importantă deoarece timpii de execuție și comportamentul anumitor componente Spark ML pot depinde de mediul folosit.

Pașii de rulare sunt următorii:

- Se deschide fișierul `project.ipynb`.
- Se verifică existența fișierului `Dry_Bean_Dataset.csv` în cazul rulării locale; În Databricks, se verifică existența tabelului `dry_bean_dataset`.
- Se rulează celulele notebook-ului în ordine, de sus în jos.
- În varianta locală prima parte a notebook-ului inițializează sesiunea Spark. În varianta cloud se folosește sesiunea Spark existentă în Databricks.
- Următoarele celule verifică schema datelor, pregătesc caracteristicile și împart datele în set de antrenare și set de test.
- Se construiesc pipeline-uri Spark ML pentru cei trei algoritmi: Decision Tree, Random Forest și Logistic Regression.
- Pentru fiecare model sunt afișate predicții, Accuracy, F1-score și timpul de antrenare.
-

Variabilele și configurațiile importante pentru reproducerea rezultatelor sunt:

- `seed = 42`
- `train/test split = 80/20`
- Decision Tree: `maxDepth = 6`
- Random Forest: `numTrees = 30`, `maxDepth = 6`
- Logistic Regression: `maxIter = 50`

- Spark local mode = local[*]
- Cloud environment = Databricks

8. Limitări și următorul pas

Au fost folosiți trei algoritmi Spark ML: Decision Tree, Random Forest și Logistic Regression. Totuși, proiectul nu include optimizare avansată de hiperparametri și nu folosește metode precum CrossValidator sau TrainValidationSplit.

Datasetul nu include probleme complexe precum text nestructurat, valori lipsă masive sau date distribuite pe mai multe surse. Această alegere a fost făcută pentru ca proiectul să fie mai ușor de urmărit și pentru ca accentul să fie pus pe construirea pipeline-ului Spark ML și pe compararea algoritmilor.

Ca următor pas, proiectul ar putea fi extins prin optimizarea hiperparametrilor, folosirea validării încrucișate, testarea pe un volum mai mare de date și analiza importanței caracteristicilor.

9. Concluzii

În acest proiect a fost construit un pipeline de clasificare folosind PySpark și Spark ML. Datasetul Dry Bean a fost încărcat, verificat și pregătit pentru antrenarea modelelor, iar coloana țintă a fost transformată într-o etichetă numerică.

Au fost evaluați trei algoritmi Spark ML: Decision Tree, Random Forest și Logistic Regression. Modelele au fost antrenate pe 10.953 de rânduri și testate pe 2.658 de rânduri, folosind aceeași împărțire train/test pentru a permite compararea rezultatelor.

Rezultatele finale sunt exprimate prin Accuracy, F1-score și timpul de antrenare. Aceste valori permit compararea celor trei algoritmi și observarea diferențelor dintre un model bazat pe un singur arbore, un model ensemble și un model liniar.

Proiectul demonstrează un flux complet și reproductibil de lucru în Spark ML, de la citirea datelor până la evaluarea și salvarea rezultatelor. În plus, rularea locală și rularea în Databricks arată că notebook-ul poate fi adaptat și executat în medii diferite.

10. Declarație AI

Pentru realizarea proiectului a fost folosit un instrument AI ca ajutor pentru restructurarea ideilor aflate în raportul redactat, dar și pentru verificarea corectitudinii codului, folosind ca model cursurile, ca acestea să corespundă cerințelor prezentate în ghidul final al proiectului.

Codul a fost rulat și verificat în notebook, iar rezultatele incluse în raport sunt proprii și provin din rularea efectivă a experimentului. Benchmark-ul, valorile de Accuracy, F1-score și timpul de antrenare nu au fost inventate, ci au fost obținute din execuția codului.

Conversație: [conversație chatGPT](#)

11. Bibliografie

[1] [Dry Bean Data Set](#)

[2] [Spark Random Forest ML](#)

[3] [Decision Tree Clasifier](#)

[4] [Logistic Regression in Apache Spark](#)

[5] Ghid final pentru proiect, Prelucrarea Volumelor Mari de Date, C13–C14.